

Laboratorio 2

Diego Stefanini — prof. Patrizio Dazzi, Luca Ferrucci, a.a. 2026-27

Indice

1	Il terminale e il filesystem (lezione 1)	3
1.1	Terminale e shell	3
1.2	Il filesystem	3
1.2.1	Percorsi assoluti e relativi	3
1.2.2	Leggere <code>ls -l</code>	4
1.3	Com'è fatto un comando	4
2	Dal sorgente al programma (lezione 1)	5
2.1	Il ciclo di lavoro	5
2.2	Compilare ed eseguire	5
2.3	Quando qualcosa va storto	5
3	Il primo programma C (lezione 1)	7
3.1	I pezzi	7
3.2	Stampare: <code>puts</code> e <code>printf</code>	7
3.2.1	Dentro un placeholder	7
3.3	Mettiamo insieme i pezzi	8
4	File, metadati e redirectione (lezione 2)	9
4.1	<code>touch</code> e i metadati temporali	9
4.2	Copia, spostamento, cancellazione	9
4.3	Wildcard: le espande la shell	9
4.4	Redirezione e pipe	10
4.5	Chi riceve gli argomenti?	10

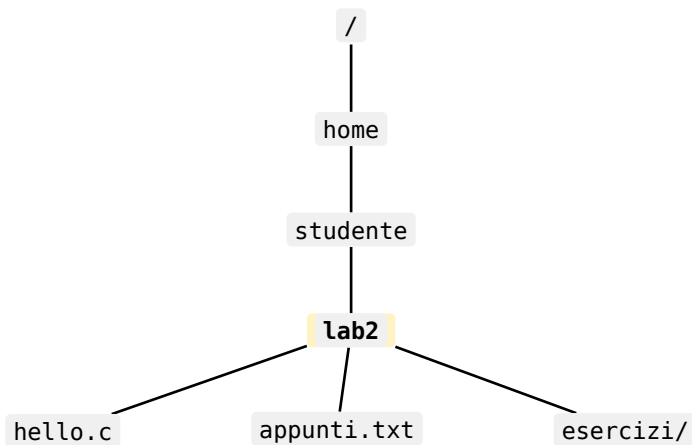
1 Il terminale e il filesystem (lezione 1)

1.1 Terminale e shell

```
$ pwd
/home/studente/lab2
$ ls
hello.c  appunti.txt
```

- **Terminale:** interfaccia testuale per dialogare con il sistema.
- **Shell** (bash, zsh): il programma dentro il terminale che interpreta i comandi.
- Il **prompt** (\$) dice che la shell aspetta un comando. Non fa parte del comando e non è sempre \$.

1.2 Il filesystem



Parte da / , la **root**. Ogni directory contiene file e altre directory; / separa un livello dal successivo.

File e directory sono oggetti diversi ma stanno nello stesso spazio dei nomi.

In giallo la **directory corrente**.

- . = la directory corrente
- .. = la directory padre

Comando	Descrizione
pwd	<i>print working directory</i> : stampa in quale cartella sei
ls	elenca il contenuto della directory corrente
ls -l	elenco in formato lungo (permessi, proprietario, dimensione, data)
cd	<i>change directory</i> : navigo in una directory
mkdir	<i>make directory</i> : crea una directory
echo \$?	restituisce il valore di ritorno dell'ultimo comando eseguito

1.2.1 Percorsi assoluti e relativi

Relativo: parte dalla directory corrente.

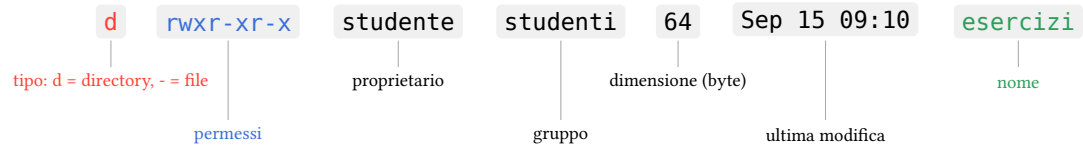
Assoluto: parte dalla root / .

```
$ cd esercizi
$ pwd
/home/studente/lab2/esercizi
```

```
$ cd /home/studente
$ pwd
/home/studente
```

```
$ pwd
/home/studente/lab2
$ cd ..      # vado nel padre
$ pwd
/home/studente
```

1.2.2 Leggere `ls -l`



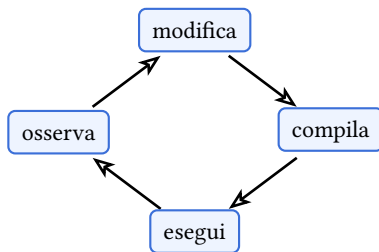
1.3 Com'è fatto un comando



Le **opzioni** modificano il comportamento del comando. Se non ricordi come si usa: `gcc --help` (sintesi rapida) oppure `man gcc` (manuale UNIX).

2 Dal sorgente al programma (lezione 1)

2.1 Il ciclo di lavoro

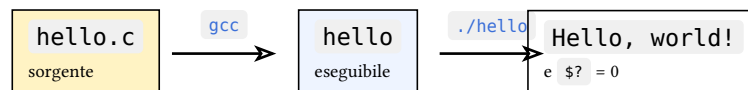


Il programma non nasce nell'editor: nasce da questo ciclo, ripetuto.

modifica ≠ compilazione ≠ esecuzione. Se cambi il sorgente e non ricompili, stai eseguendo il vecchio programma.

2.2 Compilare ed eseguire

C è un linguaggio **compilato**: il sorgente `.c` va trasformato in un file binario prima di poterlo eseguire.



```
$ gcc hello.c           # se va tutto bene non scrive niente
$ ls
a.out hello.c          # senza -o l'eseguibile si chiama a.out
$ gcc -Wall -Wextra -std=c11 hello.c -o hello # warning attivi, standard C11, nome "hello"
$ ./hello
Hello, world!
$ echo $?
0
```

Pezzo	Significato
<code>-o hello</code>	dà il nome all'eseguibile
<code>-Wall -Wextra</code>	attiva i warning: non sono rumore , spesso segnalano un errore vero
<code>-std=c11</code>	standard del C (esistono C89, C95, C99, C11, C17, C23). <code>gcc</code> = GNU C Compiler
<code>./hello</code>	«esegui <code>hello</code> che sta nella directory corrente ». La shell cerca i comandi in un elenco di directory, e quella corrente non c'è per forza
<code>echo \$?</code>	stampa il valore restituito dall'ultimo programma. 0 = terminato correttamente

2.3 Quando qualcosa va storto

```
int main(void) {
    puts("Hello, world!") // manca ;
    return 0;
}
```

```
hello.c:4:26: error: expected ';' before 'return'
```

Errore di compilazione	Errore di esecuzione
il programma non viene costruito	l'eseguibile esiste ma si comporta male o termina in modo inatteso

File, riga, messaggio: usali. Correggi **dall'alto verso il basso**: un solo errore può generare molti messaggi dopo.

3 errori banali che fanno perdere tempo

1. essere nella directory sbagliata;
2. modificare un file e compilarne un altro;
3. dimenticare di ricompilare.

Diagnosi minima

1. sono nella directory giusta?
2. il sorgente esiste?
3. ci sono errori o warning?
4. l'eseguibile è stato creato?
5. sto eseguendo quello giusto?

Routine utile: `pwd` → `ls` → `gcc ...` → `./programma` .

3 Il primo programma C (lezione 1)

3.1 I pezzi

```
#include <stdio.h>

int main(void) {
    puts("Hello!");
    return 0;
}
```

1. `#include <stdio.h>`: rende disponibili le funzioni di input/output (come `puts`)
2. `main`: da qui parte il programma. Prende niente (`void`), restituisce un intero (`int`)
3. `puts("Hello!")`: un'operazione
4. `return 0`: termina `main` e restituisce 0 al sistema operativo

Le **graffe** `{ }` delimitano un blocco. I commenti (`//` fino a fine riga, `/*` su più righe `*/`) sono per le persone: il compilatore li ignora. Indentazione e nomi sensati non sono obbligatori in C, ma il programma deve essere leggibile.

3.2 Stampare: `puts` e `printf`

Funzione	Cosa fa
<code>puts("Laboratorio 2");</code>	stampa una stringa e va a capo
<code>printf("Risultato: %d\n", 42);</code>	stringa formattata : <code>%d</code> viene sostituito dal valore. Non va a capo da sola : serve <code>\n</code>

`printf` è una funzione a **numero variabile di parametri**: il primo è la stringa di formato, gli altri i valori da inserire al posto di `%d`, `%s`, ... Il placeholder deve essere compatibile con il tipo del valore.

3.2.1 Dentro un placeholder



`printf("%08.2f", 3.14);` → `00003.14` — forma generale: `%[flags][width][.precision][length]specifier`

Interi		Altri tipi	
<code>%d</code> , <code>%i</code>	int con segno	<code>%f</code>	double
<code>%u</code>	unsigned int	<code>%e</code> , <code>%E</code>	notazione scientifica
<code>%x</code> , <code>%X</code>	esadecimale	<code>%g</code> , <code>%G</code>	formato compatto
<code>%o</code>	ottale	<code>%c</code>	carattere
<code>%ld</code>	long	<code>%s</code>	stringa
<code>%lld</code>	long long	<code>%p</code>	puntatore
<code>%zu</code>	size_t	<code>%%</code>	il carattere %

3.3 Mettiamo insieme i pezzi

```
#include <stdio.h>

int main(void) {
    int x = 21;                // una
    variabile

    puts("Laboratorio 2");
    puts("Lezione 1");
    printf("2 * %d = %d\n", x, 2 * x); // due
    placeholder

    return 0;
}
```

```
$ gcc -Wall -Wextra -std=c11
lezione01.c -o lezione01
$ ./lezione01
Laboratorio 2
Lezione 1
2 * 21 = 42
```


4 File, metadati e redirectione (lezione 2)

4.1 touch e i metadati temporali

`touch` : se il file non esiste lo crea, se esiste ne aggiorna la data di modifica. `stat` mostra le informazioni su un file, tra cui i tre tempi:

Metadato	Significato
<code>atime</code>	<i>access time</i> : ultimo accesso al file
<code>mtime</code>	<i>modification time</i> : ultima modifica del contenuto
<code>ctime</code>	<i>change time</i> : ultima modifica dei metadati

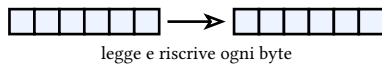
Chi aggiorna cosa (✓ = portato all'ora corrente):

Operazione	<code>atime</code>	<code>mtime</code>	<code>ctime</code>
<code>touch file</code>	✓	✓	✓ (cambiare i tempi è già una modifica dei metadati)
<code>cp</code> : destinazione	✓ solo se creato	✓	✓
<code>cp -p</code> : destinazione	copiato dal sorgente	copiato dal sorgente	✓
<code>cp</code> : sorgente	✓ può, per la lettura	—	—

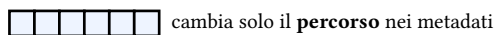
4.2 Copia, spostamento, cancellazione

```
cp /tmp/ciao.txt ~/hello.txt # copia ciao.txt da /tmp nella home, rinominandolo hello.txt
```

copia — $O(N)$



sposta (stesso filesystem) — $O(1)$

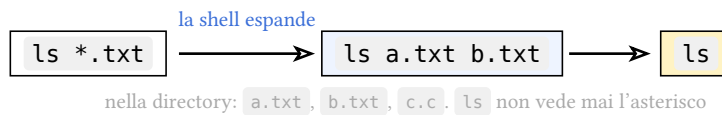


Operazione	Costo
copia	$O(N)$
sposta, stesso filesystem	$O(1)$
sposta, altro filesystem	$O(N)$: copia + cancella

Comando	Descrizione
<code>rm</code>	cancella un file
<code>rmdir</code>	cancella una directory vuota
<code>rm -r</code>	cancella una directory con tutto il contenuto (file e sottodirectory)
<code>cat</code>	stampa il contenuto di un file su stdout
<code>less</code>	come <code>cat</code> , ma si scorre avanti e indietro e si cerca
<code>head</code> / <code>tail</code>	prime / ultime 10 righe. <code>head -n 5 file.txt</code> = prime 5

4.3 Wildcard: le espande la shell

- `*` = qualsiasi sequenza di caratteri: `*.txt` = tutti i file che finiscono in `.txt`
- `?` = **un solo** carattere qualsiasi: `prova?.txt` va bene per `prova1.txt`, `provaA.txt`



4.4 Redirezione e pipe

Ogni programma legge dallo **standard input** (di solito la tastiera) e scrive sullo **standard output** (di solito lo schermo). La shell può ricollegarli:

